Advanced Data Structure

[Subject Code: 25CSH-202]

Project Report

Project name: Graph Visualizer - Design and Development of an Interactive Graph & Tree Visualizer Using C++ and SFML.

Submitted by:	**Submitted to:**
Name & UID: Anurag Rana (25BCS10311)	**Faculty Name:** Mr. MD Tauseef
Name & UID: Phalgun (25BCS12747)	**Date of Submission:** 13-11-2026
Name & UID: Anurag Singh (25BCS11736)	
Name & UID: Tanvi Anand (25BCS11368)	

ABSTRACT

The **Graph Visualizer** is an interactive desktop application developed using **C++ and SFML 3** to demonstrate and visualize fundamental concepts of Data Structures and Algorithms. The main objective of this project is to make complex algorithms easier to understand through graphical representations and step-by-step animations rather than relying only on theoretical explanations.

The application consists of two major modules: **Graph Visualization** and **Binary Search Tree (BST) Visualization**. The Graph Visualization module allows users to create nodes, establish weighted edges, modify graph structures, and visualize important graph algorithms, including Dijkstra's Shortest Path Algorithm, Prim's Minimum Spanning Tree Algorithm, and Kruskal's Minimum Spanning Tree Algorithm. The BST module provides interactive operations such as insertion, searching, deletion, and tree traversals, including inorder, preorder, postorder, and level-order traversal. It also demonstrates tree properties such as height and diameter.

The project uses various data structures, including vectors, queues, priority queues, and Disjoint Set Union (DSU), to implement the algorithms efficiently. SFML is used to create the graphical user interface, handle user interactions, and provide visual animations of algorithm execution.

Overall, this project aims to bridge the gap between theoretical knowledge and practical implementation by providing an interactive learning environment. It helps users understand algorithmic logic, data structure operations, and their execution processes in a simple and engaging manner.

Table of Contents

S. No.	Contents	Page No.
1.	Introduction	4
2.	Problem Statement	5
3.	Objectives	7
4.	Technologies Used	8
5.	System Requirements	9
6.	System Design and Architecture	11
7.	Modules Description	14
8.	Algorithm Implementation	18
9.	Data Structures Used	21
10.	Time and Space Complexity Analysis	22
11.	Implementation and Working	24
12.	Testing and Results	25
13.	Application Screenshots	26
14.	Limitations	30
15.	Future Scope	30
16.	Conclusion	32
17.	References	33

1. INTRODUCTION

Data Structures and Algorithms (DSA) are fundamental concepts of computer science that play an important role in organizing data and solving computational problems efficiently. Understanding these concepts requires both theoretical knowledge and practical implementation. However, learning complex algorithms through textbooks and theoretical explanations alone can sometimes be difficult.

The **Graph Visualizer** is an interactive desktop application developed using **C++ and SFML 3**. It is designed to provide a visual understanding of different data structures and algorithms through graphical representations and step-by-step animations. The application allows users to interact with data structures and observe how different operations are performed.

The project consists of two major modules: **Graph Visualization** and **Binary Search Tree (BST) Visualization**.

The Graph Visualization module allows users to create nodes, connect them through weighted edges, move or delete nodes, and perform various graph algorithms. It includes Dijkstra's Shortest Path Algorithm, Prim's Minimum Spanning Tree Algorithm, and Kruskal's Minimum Spanning Tree Algorithm. These algorithms help users understand shortest-path calculations and minimum spanning tree construction.

The Binary Search Tree module provides functionality for inserting, searching, and deleting nodes. It also supports different tree traversal techniques, including inorder, preorder, postorder, and level-order traversal. Additionally, users can visualize important tree properties such as height and diameter.

The application uses C++ Standard Template Library (STL) components such as vectors, queues, priority queues, and Disjoint Set Union (DSU) to implement different operations. SFML is used to develop the graphical user interface, manage user interactions, and display algorithm execution through animations.

The primary aim of this project is to make learning Data Structures and Algorithms more interactive, practical, and understandable. By combining programming concepts with graphical visualization, Graph Visualizer provides an effective environment for exploring algorithmic operations and understanding their working mechanisms.

2. PROBLEM STATEMENT

Data Structures and Algorithms (DSA) are fundamental concepts in computer science that help in organizing data and solving computational problems efficiently. However, understanding the internal working of different data structures and algorithms can be challenging when they are studied only through theoretical explanations, textbooks, and static diagrams.

Graph algorithms such as Dijkstra's Shortest Path, Prim's Minimum Spanning Tree, and Kruskal's Minimum Spanning Tree involve multiple steps, comparisons, and decision-making processes. Understanding how nodes are selected, how edges are evaluated, and how the final result is obtained requires careful observation of each operation. Similarly, Binary Search Tree operations such as insertion, deletion, searching, and traversal involve structural changes that may be difficult to follow without proper visualization.

Another major challenge is the lack of an interactive platform where users can independently create data structures, perform operations, and observe the results during execution. Manual tracing of algorithms can also become time-consuming, especially when working with larger or more complex inputs.

To address these challenges, the **Graph Visualizer** project aims to develop an interactive desktop application using **C++ and SFML 3**. The application provides a graphical environment where users can create and modify weighted graphs, execute important graph algorithms, and observe their execution through visual animations. It also includes a separate Binary Search Tree module to demonstrate different tree operations and traversal techniques.

The major problems addressed by this project are:

- Difficulty in understanding the step-by-step execution of complex algorithms.
- Lack of interactive graphical representation of data structures.
- Challenges in understanding shortest-path and minimum spanning tree calculations.
- Difficulty in visualizing structural changes during Binary Search Tree insertion and deletion.
- Time-consuming manual tracing of algorithm execution.
- Limited opportunities to experiment with different graph and tree configurations.
- Need for a user-friendly learning environment that connects theoretical concepts with practical programming.

The proposed Graph Visualizer provides a practical and interactive solution to these challenges by combining algorithm implementation with graphical visualization, making fundamental Data Structures and Algorithms easier to understand, explore, and analyze.

3. OBJECTIVES

The primary objective of the **Graph Visualizer** project is to design and develop an interactive desktop application using C++ and SFML 3 that provides a visual understanding of fundamental Data Structures and Algorithms. The project aims to make complex algorithmic concepts easier to understand by representing their execution through graphical elements and animations.

The application focuses on the visualization and practical implementation of important graph algorithms, including Dijkstra's Shortest Path Algorithm, Prim's Minimum Spanning Tree Algorithm, and Kruskal's Minimum Spanning Tree Algorithm. It allows users to create and modify weighted graphs by adding nodes, connecting them through edges, moving nodes, and deleting unwanted elements. This helps users understand graph structures and observe how different algorithms process nodes and edges to produce results.

Another major objective is to develop a Binary Search Tree (BST) module that demonstrates fundamental tree operations such as insertion, searching, deletion, and clearing the tree. The application also provides visualization of different traversal techniques, including inorder, preorder, postorder, and level-order traversal. Additionally, it calculates important tree properties such as height and diameter.

The project also aims to implement efficient programming techniques using C++ Standard Template Library (STL) components, including vectors, queues, priority queues, and Disjoint Set Union (DSU). Through the use of SFML 3, the application provides a graphical user interface that supports user interaction and displays algorithm execution through animations.

Overall, the objective of this project is to bridge the gap between theoretical concepts and practical implementation by creating an interactive learning platform. It encourages experimentation, improves problem-solving skills, and helps users develop a deeper understanding of data structures, algorithmic operations, and their applications in computer science.

4. TECHNOLOGIES USED

The Graph Visualizer project is developed using C++ and supporting technologies to create an interactive desktop application for visualizing Data Structures and Algorithms. Different programming tools and libraries are integrated to handle algorithm implementation, graphical representation, user interaction, and project compilation.

C++17 is used as the primary programming language for developing the application. It provides features such as object-oriented programming, efficient memory management, and support for implementing different data structures and algorithms.

SFML 3 (Simple and Fast Multimedia Library) is used to develop the graphical interface of the application. It helps in creating the application window, drawing graph nodes and edges, displaying text, handling mouse and keyboard events, and implementing animations for visualizing algorithm execution.

CMake is used as the build configuration system. It manages project settings, specifies the required C++ standard, locates external libraries such as SFML, and organizes the compilation and linking process.

The project also utilizes the **C++ Standard Template Library (STL)** to implement various data structures and operations. Vectors are used to store graph nodes and edges, priority queues support efficient node selection in Dijkstra's algorithm, and queues are used for level-order traversal of the Binary Search Tree. Standard sorting operations are also used in Kruskal's algorithm to arrange graph edges according to their weights.

Additionally, **Disjoint Set Union (DSU)** is implemented to manage connected components and detect cycles during the execution of Kruskal's Minimum Spanning Tree algorithm.

The combination of these technologies provides the foundation for developing an interactive, efficient, and visually engaging application that demonstrates the practical working of fundamental Data Structures and Algorithms.

4.1 Technology Summary

Technology	Purpose
C++17	Core programming and algorithm implementation

SFML 3	Graphical interface and animations
CMake	Project configuration and build management
C++ STL	Implementation of data structures and operations
Disjoint Set Union	Cycle detection in Kruskal's algorithm

5. SYSTEM REQUIREMENTS

The Graph Visualizer is a desktop-based application developed using C++ and SFML 3. It requires a compatible computer system, a C++ compiler, and the necessary libraries to build and execute the application. The following hardware and software requirements are recommended for running the project smoothly.

5.1 Hardware Requirements

Component	Recommended Requirement
Processor	Intel Core i3 or equivalent
RAM	Minimum 4 GB
Storage	500 MB of available space
Display	1366 × 768 resolution or higher
Input Devices	Keyboard and Mouse
Graphics	Basic graphics support compatible with SFML

5.2 Software Requirements

Software	Requirement
Operating System	Windows 10/11 or compatible OS
Programming Language	C++
Language Standard	C++17
Graphics Library	SFML 3
Build System	CMake 3.28 or higher
Compiler	GCC, MinGW, MSVC, or compatible C++ compiler
Code Editor	Visual Studio Code or equivalent
Required SFML Modules	Graphics, Window, System

5.3 Development Environment

The application is developed using C++17, which provides object-oriented programming features, standard library support, and efficient memory management. SFML 3 is used to create the graphical interface, render nodes and edges, handle user interactions, and display algorithm animations.

CMake is used as the build configuration system to manage project compilation and link the required SFML libraries. The C++ Standard Template Library (STL) provides useful components such as vectors, queues, priority queues, and sorting operations for implementing the algorithms.

5.4 Functional Requirements

The system should provide the following functionalities:

- Allow users to create, move, and delete graph nodes.
- Allow users to create weighted edges between nodes.
- Execute Dijkstra's algorithm to find the shortest path.
- Execute Prim's and Kruskal's algorithms to generate Minimum Spanning Trees.
- Support insertion, searching, and deletion in a Binary Search Tree.
- Display different tree traversal techniques.
- Calculate the height and diameter of a Binary Search Tree.
- Provide graphical animations to represent algorithm execution.

5.5 Non-Functional Requirements

The application should provide a user-friendly graphical interface, responsive interaction, clear visualization, and reliable execution of supported operations. It should also maintain readable code structure and allow future extension with additional data structures and algorithms.

6. SYSTEM DESIGN AND ARCHITECTURE

The system design of the Graph Visualizer describes how different components of the application work together to provide an interactive environment for visualizing data structures and algorithms. The application follows a modular design in which graph operations, tree operations, algorithm execution, user interaction, and graphical rendering are handled through separate functional components.

6.1 System Architecture

The application is organized into the following major layers:

1. User Interface Layer

This layer provides the graphical interface through which users interact with the application. It includes the navigation panel, tool panel, and visualization area. Users can select different workspaces, enter values, create graph elements, and execute algorithms.

2. Graph Management Module

This module manages the graph structure. It allows users to add and delete nodes, create weighted edges, and change node positions. Graph information is maintained using collections of nodes and edges.

3. Graph Algorithm Module

This module implements the graph-related algorithms:

- Dijkstra's Shortest Path Algorithm
- Prim's Minimum Spanning Tree Algorithm
- Kruskal's Minimum Spanning Tree Algorithm

It processes the graph data and determines the required paths or spanning trees.

4. Binary Search Tree Module

This module handles all BST-related operations, including insertion, searching, deletion, and traversal. It also calculates the height and diameter of the tree.

5. Visualization and Animation Module

This module displays graph nodes, edges, weights, and tree structures. It highlights selected nodes and edges during algorithm execution to help users understand each step visually.

6. Event Handling Module

This module processes mouse and keyboard inputs. It handles actions such as selecting options, clicking buttons, dragging nodes, and entering data.

6.2 Overall System Workflow

The general working process of the application is as follows:

- The application starts and initializes the graphical window.
- The user selects either the Graph Editor or Binary Search Tree workspace.
- The user performs operations through the available interface.
- The application processes the input using the appropriate data structure or algorithm.
- The visualization module displays the results through graphical elements and animations.
- The user can continue performing operations or switch to another workspace.

6.3 Graph Data Representation

The graph is represented using two main components: nodes and edges.

- **Nodes:** Each node stores its name and position on the screen.
- **Edges:** Each edge stores the indices of its two connected nodes and the associated weight.

The graph is undirected and weighted. The application uses collections to maintain the graph's nodes and edges.

6.4 Binary Search Tree Representation

The Binary Search Tree is represented using tree nodes containing a value and references to their left and right children. The application uses index-based representation, where `-1` indicates that a child does not exist.

This representation supports insertion, searching, deletion, traversal, and calculation of tree properties.

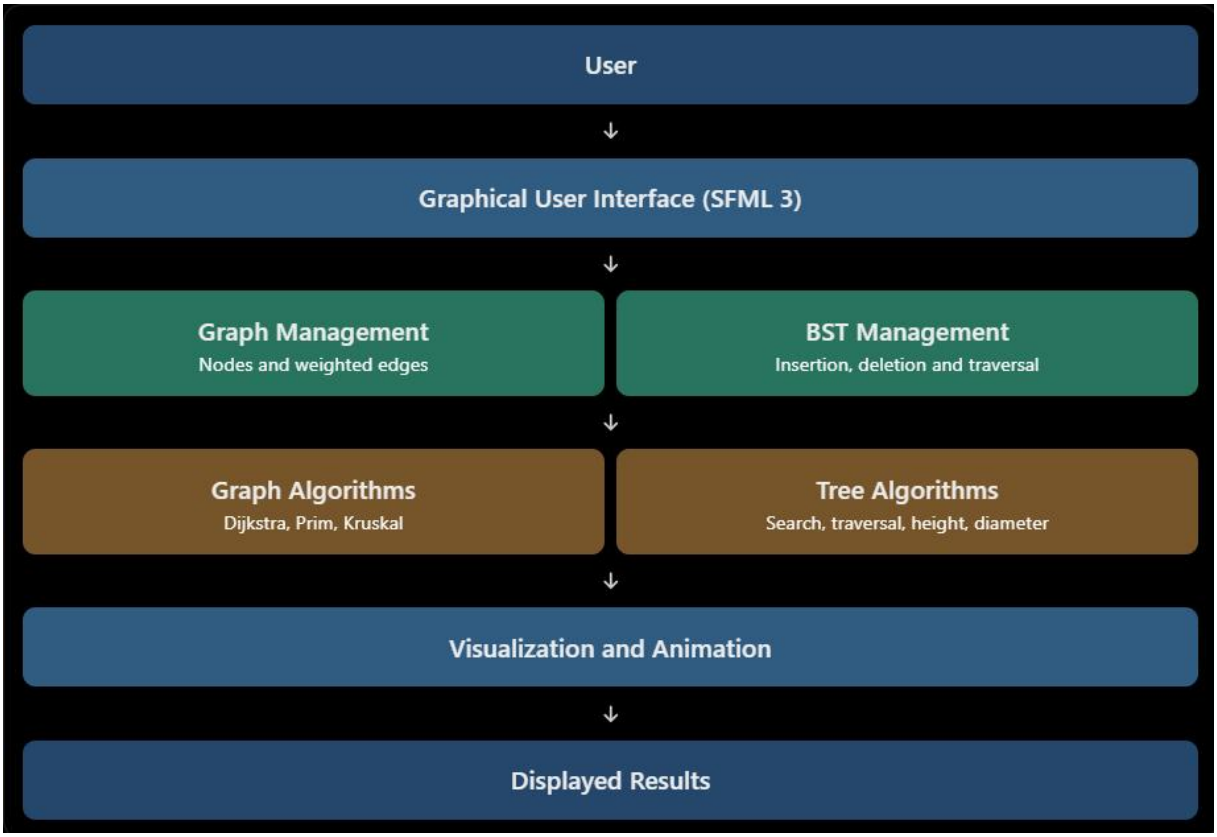
6.5 Graphical Interface Design

The application uses a dark-themed, three-column dashboard layout:

- **Navigation Panel:** Provides access to different workspaces.
- **Toolkit Panel:** Contains the controls and operations available in the selected workspace.
- **Visualization Canvas:** Displays the graph or tree structure and the results of algorithm execution.

SFML 3 is responsible for rendering graphical elements and managing window events. The application window uses a resolution of 1450 × 800 pixels and is configured to run at 60 frames per second.

6.6 Architecture Diagram



Screenshot: Architecture Diagram

7. MODULES DESCRIPTION

The Graph Visualizer application is divided into different modules to provide a structured and interactive platform for understanding data structures and algorithms. Each module performs specific operations and contributes to the overall functionality of the application.

7.1 Graph Editor Module

The Graph Editor module allows users to create and modify weighted undirected graphs through a graphical interface. Users can place nodes on the visualization canvas, connect them using weighted edges, and modify the graph structure.

The major functionalities include:

- Adding nodes with unique names.
- Creating weighted edges between two nodes.
- Moving nodes by dragging them across the canvas.
- Deleting nodes along with their connected edges.
- Selecting source and destination nodes for algorithm execution.

This module provides the basic graph structure required for implementing graph algorithms.

7.2 Dijkstra's Shortest Path Module

This module implements Dijkstra's algorithm to find the shortest path between a selected source node and destination node in a weighted graph.

The algorithm calculates the minimum distance from the source node to other reachable nodes. A priority queue is used to process nodes according to their current shortest distance. The application highlights the selected path and displays the result through graphical animation.

The algorithm works with non-negative edge weights.

7.3 Prim's Minimum Spanning Tree Module

Prim's algorithm is used to construct a Minimum Spanning Tree (MST) of a connected, weighted, undirected graph.

The algorithm starts from the first node and repeatedly selects the minimum-weight edge that connects an already included node to a node outside the current tree. The selected edges are highlighted on the visualization canvas.

The final result connects all vertices without forming cycles and has the minimum possible total edge weight.

7.4 Kruskal's Minimum Spanning Tree Module

Kruskal's algorithm constructs a Minimum Spanning Tree by selecting edges in increasing order of their weights.

The algorithm first sorts all graph edges according to their weights. It then selects an edge if it does not create a cycle. A Disjoint Set Union (DSU) data structure is used to detect cycles efficiently.

The selected edges are highlighted, and the application displays the resulting Minimum Spanning Tree.

7.5 Binary Search Tree Module

The Binary Search Tree module allows users to perform basic operations on a Binary Search Tree through an interactive graphical interface.

The main operations include:

- **Insertion:** Adds a new value while maintaining the BST property.
- **Searching:** Locates a particular value in the tree.
- **Deletion:** Removes a selected value while maintaining the BST structure.
- **Clear Tree:** Removes all existing nodes from the tree.

Duplicate values are ignored during insertion.

7.6 Tree Traversal Module

The application supports four major tree traversal techniques to demonstrate different ways of visiting nodes.

- **Inorder Traversal:** Visits the left subtree, root node, and right subtree.

- **Preorder Traversal:** Visits the root node, left subtree, and right subtree.
- **Postorder Traversal:** Visits the left subtree, right subtree, and root node.
- **Level-Order Traversal:** Visits nodes level by level using a queue.

During traversal, the application highlights the visited nodes through animation, allowing users to understand the sequence of execution.

7.7 Tree Properties Module

This module calculates two important properties of the Binary Search Tree:

- **Height:** Represents the length of the longest path from the root node to a leaf node, according to the implementation's height convention.
- **Diameter:** Represents the longest path between any two nodes in the tree.

These properties help users understand the structure and performance characteristics of a Binary Search Tree.

7.8 Visualization and Animation Module

The visualization module connects algorithm execution with graphical representation. It displays nodes, edges, weights, and tree structures on the screen.

During execution, selected nodes and edges are highlighted to represent the algorithm's progress. This makes the application more interactive and helps users understand how different operations modify or process data structures.

8. ALGORITHM IMPLEMENTATION

The Graph Visualizer implements graph and tree algorithms using C++ and STL. These algorithms perform operations such as shortest path calculation, Minimum Spanning Tree generation, insertion, deletion, searching, and traversal. Their execution is represented through graphical animations.

8.1 Dijkstra's Shortest Path Algorithm

Dijkstra's algorithm finds the shortest path between a source vertex and other reachable vertices in a weighted graph with non-negative edge weights.

Working:

- Initialize the source distance to zero and all other distances to infinity.
- Insert the source into a priority queue.
- Extract the vertex with the minimum distance.
- Examine its connected edges and calculate new distances.
- Update distances and predecessor information if a shorter path is found.
- Reconstruct and highlight the shortest path.

8.2 Prim's Minimum Spanning Tree Algorithm

Prim's algorithm constructs a Minimum Spanning Tree (MST) of a connected, weighted, undirected graph.

Working:

- Start from the first vertex.
- Select the minimum-weight edge connecting an included vertex to an unvisited vertex.
- Add the selected edge and new vertex to the MST.
- Repeat until all vertices are included or no connecting edge exists.

8.3 Kruskal's Minimum Spanning Tree Algorithm

Kruskal's algorithm builds an MST by selecting edges in ascending order of weight. It uses Disjoint Set Union (DSU) for cycle detection.

Working:

- Sort all edges by weight.
- Initialize each vertex as a separate set.
- Select the minimum-weight edge.
- If its endpoints belong to different sets, include it and merge the sets.
- Reject edges that create cycles.
- Continue until V-1 edges are selected or all edges are examined.

8.4 Binary Search Tree Insertion

Insertion adds a new value while maintaining the BST property.

Working:

- If the tree is empty, create the root.
- Compare the new value with the current node.
- Move left if smaller and right if greater.
- Insert the value at the empty position.
- Ignore duplicate values.

8.5 Binary Search Tree Searching

Searching locates a particular value in the BST.

Working:

- Start from the root.
- Compare the target with the current node.
- If equal, the value is found.
- If smaller, move left; otherwise, move right.
- Continue until found or an empty position is reached.

8.6 Binary Search Tree Deletion

Deletion removes a node while maintaining the BST property. It involves three cases:

- **Leaf Node:** Remove the node directly.
- **One Child:** Replace the node with its child.
- **Two Children:** Replace the node with its inorder successor and remove the successor from its original position.

8.7 Tree Traversal Algorithms

The application supports four major traversal techniques:

Traversal	Visiting Order
Inorder	Left – Root – Right
Preorder	Root – Left – Right
Postorder	Left – Right – Root
Level-Order	Level by Level

Inorder, preorder, and postorder use recursion, while level-order traversal uses a queue.

8.8 Height and Diameter Calculation

Height: Calculates the longest downward path from the root to a leaf node.

Diameter: Calculates the longest path between any two nodes in the tree.

Both properties are calculated using recursive operations.

8.9 Algorithm Visualization

The application uses graphical animations to demonstrate algorithm execution. Selected nodes and edges are highlighted to represent the progress of graph algorithms and tree operations. This helps users understand the working and results of different algorithms interactively.

9. DATA STRUCTURES USED

The Graph Visualizer uses different data structures to store and process information efficiently. These structures support graph algorithms and Binary Search Tree operations.

9.1 Vector

A vector is a dynamic array provided by the C++ STL.

Uses:

- Stores graph nodes and edges.
- Maintains distances and predecessor information.
- Stores tree nodes and selected MST edges.

9.2 Priority Queue

A priority queue processes elements according to their priority.

Use: It is used in Dijkstra's algorithm to select the vertex with the minimum current distance.

9.3 Queue

A queue follows the FIFO (First In, First Out) principle.

Use: It is used in level-order traversal to visit Binary Search Tree nodes level by level.

9.4 Disjoint Set Union (DSU)

DSU manages separate groups of elements through two operations: Find and Union.

Use: It is used in Kruskal's algorithm to detect cycles and combine different sets. Path compression and union by rank improve its efficiency.

9.5 Binary Search Tree

A BST is a hierarchical data structure in which smaller values are stored in the left subtree and greater values in the right subtree.

Use: It supports insertion, searching, deletion, and traversal operations in the application.

9.6 Recursion

Recursion is a technique in which a function calls itself to solve smaller parts of a problem.

Use: It is used for BST insertion, deletion, searching, traversal, height, and diameter calculations.

9.7 Sorting

Sorting arranges elements in a specific order.

Use: The C++ STL sorting function arranges graph edges in ascending order of weight before Kruskal's algorithm executes.

9.8 Summary

Data Structure	Project Application
Vector	Storing graph and tree data
Priority Queue	Dijkstra's algorithm
Queue	Level-order traversal
DSU	Cycle detection in Kruskal's algorithm
BST	Tree operations
Recursion	Tree operations and calculations
Sorting	Ordering graph edges

These data structures provide efficient ways to organize and process information, making the implementation of graph and tree algorithms easier and more structured.

10. TIME AND SPACE COMPLEXITY ANALYSIS

Time complexity measures the execution time of an algorithm, while space complexity measures the memory required during its execution.

The following table shows the complexity of the algorithms used in the Graph Visualizer.

10.1 Time Complexity

Algorithm	Time Complexity
Dijkstra's Algorithm*	Implementation-dependent
Prim's Algorithm	O(V × E)
Kruskal's Algorithm	O(E log E)
BST Insertion	O(log N) average, O(N) worst
BST Searching	O(log N) average, O(N) worst
BST Deletion	O(log N) average, O(N) worst
Tree Traversals	O(N)
Height Calculation	O(N)
Diameter Calculation	O(N)

The project's Dijkstra implementation scans the edge list for each processed vertex, so its complexity is higher than the standard adjacency-list implementation.

10.2 Space Complexity

Data Structure / Algorithm	Space Complexity
Graph Storage	O(V + E)
Dijkstra's Algorithm	O(V)
Kruskal's Algorithm	O(V + E)
Binary Search Tree	O(N)
Tree Traversal	O(N)

Where:

- **V** = Number of vertices.
- **E** = Number of edges.
- **N** = Number of tree nodes.

10.3 Conclusion

The application uses appropriate data structures and algorithms to perform graph and tree operations. Its complexity depends on the selected algorithm and the structure of the input data.

11. IMPLEMENTATION AND WORKING

The Graph Visualizer is developed using C++17 and SFML 3. It provides an interactive graphical interface with a 1450 × 800 window and a 60 FPS display.

11.1 Application Initialization

The application initializes the SFML window, graphical components, fonts, and required data structures.

11.2 Graph Editor

Users can add nodes, connect them using weighted edges, drag nodes to change their positions, and delete unwanted nodes or connections.

11.3 Graph Algorithm Execution

Users can execute Dijkstra's, Prim's, and Kruskal's algorithms. The application processes the graph and highlights the resulting shortest path or Minimum Spanning Tree.

11.4 Binary Search Tree Operations

Users can insert, search, delete, and clear tree nodes. The application also supports four traversal methods and calculates tree height and diameter.

11.5 Graphical Animation

During execution, selected nodes and edges are highlighted. Tree traversal also displays visited nodes through animations at regular intervals.

11.6 Result Display

The final output is displayed directly on the visualization canvas, allowing users to observe the results of different operations.

This implementation provides an interactive environment for understanding graph algorithms and Binary Search Tree operations.

12. TESTING AND RESULTS

Testing is important to ensure that the application's features work according to their expected functionality. The following test cases can be used to verify the Graph Visualizer.

12.1 Test Cases

Test Case	Expected Result
Add graph node	Node appears on canvas
Create weighted edge	Edge connects selected nodes
Delete node	Node and connected edges are removed
Execute Dijkstra	Shortest path is highlighted
Execute Prim	Minimum Spanning Tree is displayed
Execute Kruskal	MST is displayed without cycles
Insert BST value	Node is added correctly
Search BST value	Element is found or reported absent
Delete BST value	Node is removed correctly
Tree traversal	Nodes are visited in the selected order
Calculate height and diameter	Tree properties are displayed

12.2 Expected Results

The application is designed to provide the following results:

- Correct visualization of graph nodes and weighted edges.
- Identification of shortest paths using Dijkstra's algorithm.
- Generation of Minimum Spanning Trees using Prim's and Kruskal's algorithms.
- Correct execution of BST insertion, searching, and deletion.
- Visualization of all four tree traversal techniques.
- Calculation of tree height and diameter.

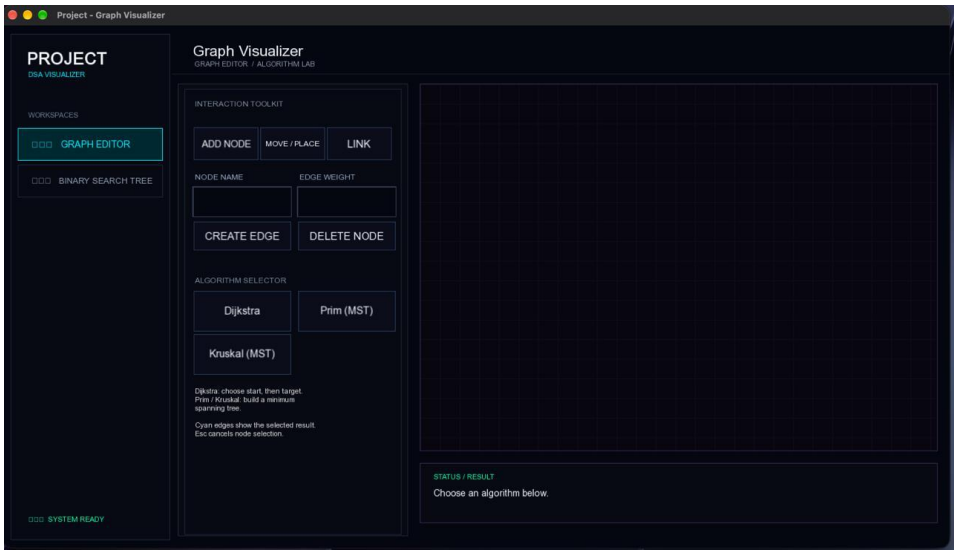
12.3 Overall Outcome

The test cases cover the major functionalities of the application. They provide a basis for checking algorithm correctness, graphical interaction, and expected output.

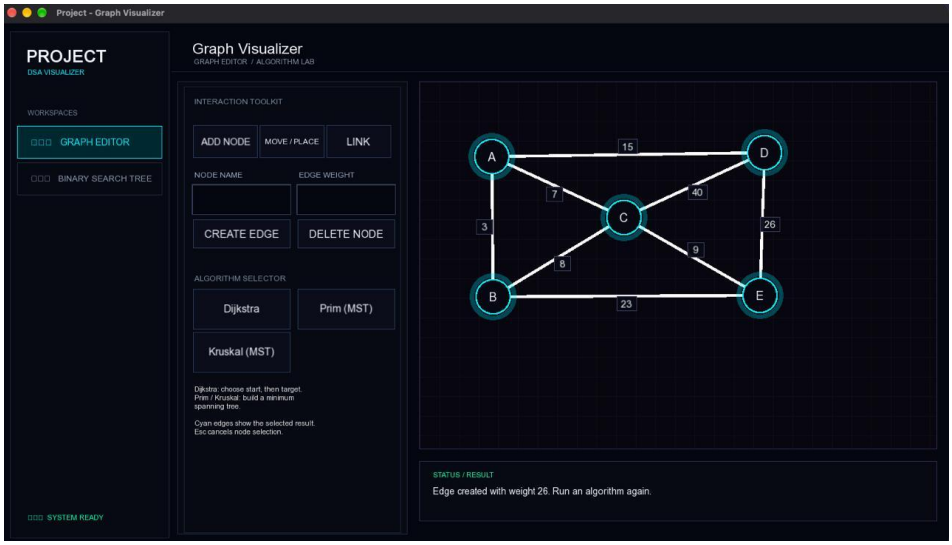
13. APPLICATION SCREENSHOTS

Screenshots of the Graph Visualizer demonstrate the graphical interface and working of its different modules.

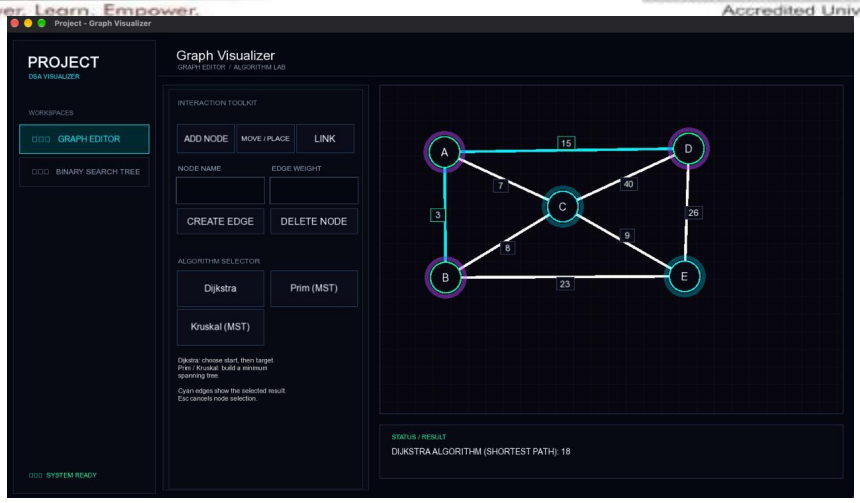
Include the following screenshots in this section:



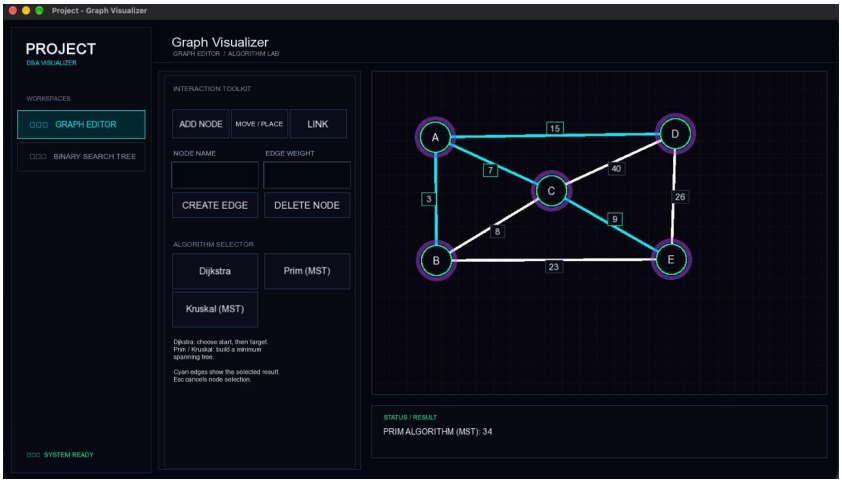
Main Interface: Overall application dashboard.



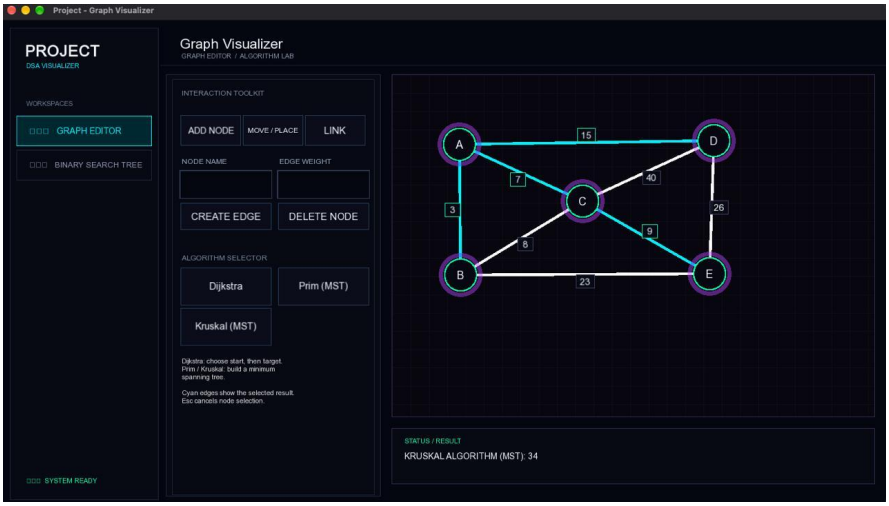
Graph Creation: Graph containing multiple nodes and weighted edges.



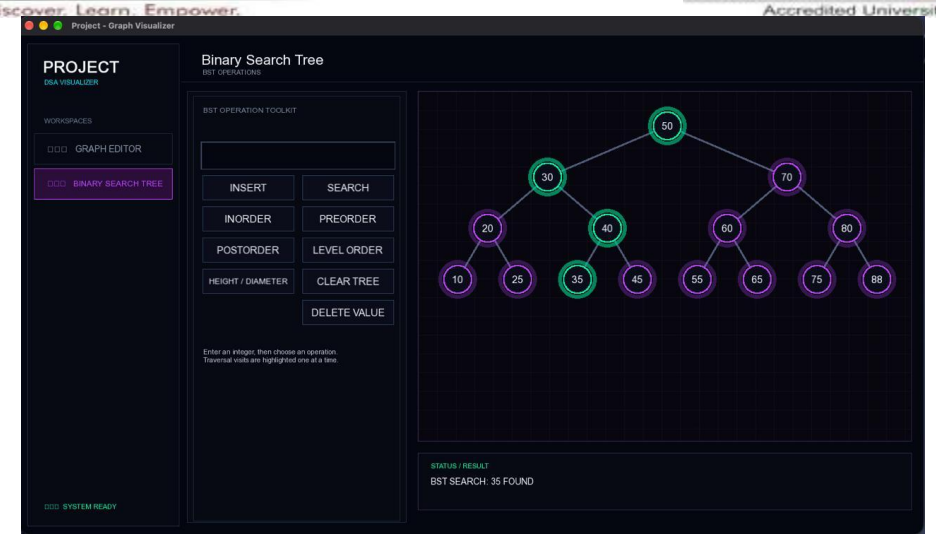
Dijkstra's Algorithm: Highlighted shortest path.



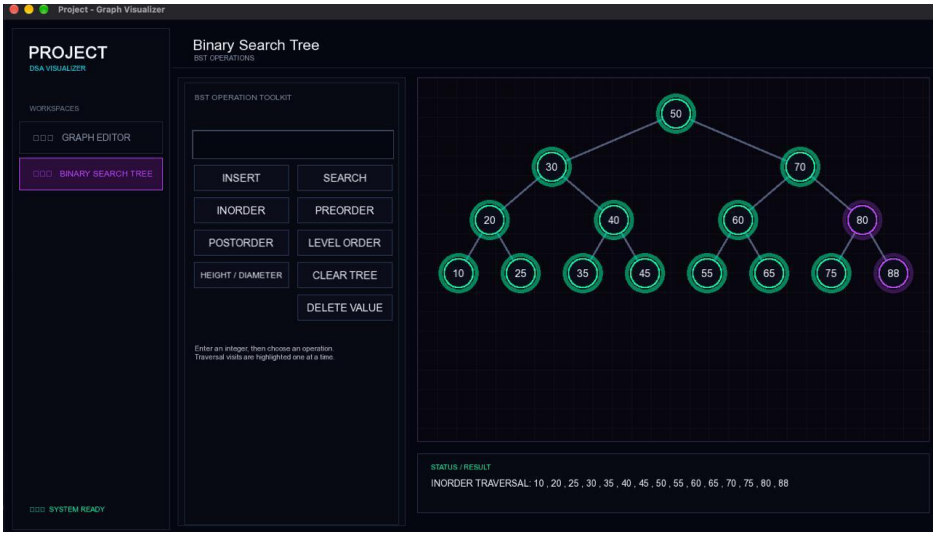
Prim's Algorithm: Minimum Spanning Tree visualization.



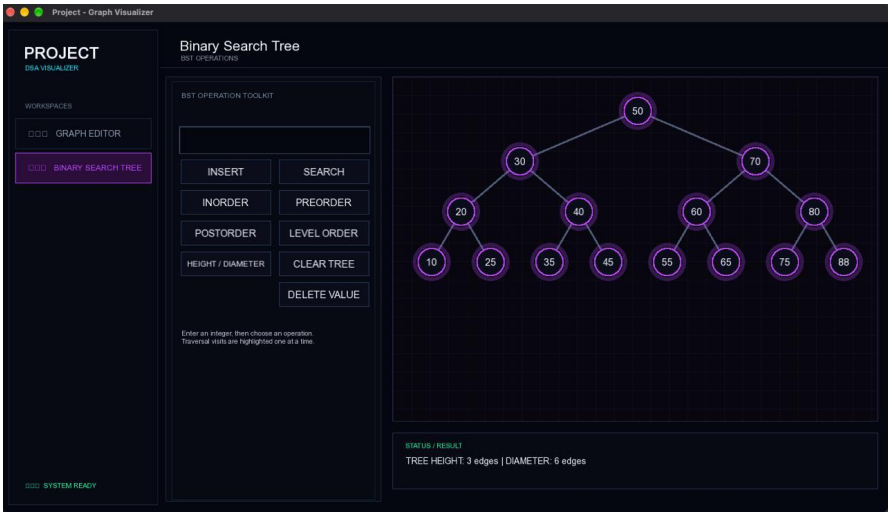
Kruskal's Algorithm: Selected MST edges.



BST Operations: Binary Search Tree after insertion.(e.g search operation)



Tree Traversal: Visualization of traversal operations.(e.g Inorder traversal)



Tree Properties: Height and diameter calculation.

14. LIMITATIONS

Although the Graph Visualizer provides several useful features, it has certain limitations:

- **Limited Algorithms:** Supports selected graph algorithms and BST operations only.
- **Non-Negative Weights:** Dijkstra's algorithm does not support negative edge weights.
- **Undirected Graph:** Graph operations are designed for undirected graphs.
- **Unbalanced BST:** The tree does not automatically balance itself after insertion or deletion.
- **No Data Persistence:** Graphs and trees are not permanently saved after closing the application.
- **Limited Customization:** Animation speed and visualization options are limited.

These limitations provide opportunities for future improvements and additional functionality.

15. FUTURE SCOPE

The Graph Visualizer has significant potential for future development. Although the current application supports important graph algorithms and Binary Search Tree operations, several additional features can be introduced to improve its functionality, performance, and educational value.

15.1 Addition of More Algorithms

The application can be extended by implementing additional graph algorithms such as Breadth-First Search (BFS), Depth-First Search (DFS), Bellman-Ford, Floyd-Warshall, and A* algorithms. These additions would help users understand different approaches to graph traversal and shortest-path problems.

15.2 Advanced Data Structures

More data structures, such as AVL Trees, Red-Black Trees, Heaps, and Tries, can be incorporated. This would allow users to compare different data structures and understand their operations and performance.

15.3 Advanced Animation Controls

Features such as pause, resume, restart, and animation speed adjustment can be introduced. These controls would allow users to study algorithm execution at their own pace.

15.4 Save and Load Functionality

A save and load feature can be developed to allow users to store their graph and tree structures and access them later without recreating them.

15.5 Step-by-Step Algorithm Explanation

The application can display pseudocode, execution steps, and explanations alongside the graphical visualization. This would make complex algorithms easier to understand, especially for beginners.

15.6 Improved Graph Support

Support for directed graphs, larger graph structures, and additional graph operations can be introduced to make the application more flexible and useful for advanced learning.

15.7 Performance Optimization

The application can be further optimized to handle larger datasets efficiently by improving graph representation and algorithm implementation.

Overall, these improvements can transform the Graph Visualizer into a more comprehensive learning platform that combines theoretical concepts, practical implementation, and interactive visualization.

16. CONCLUSION

The Graph Visualizer project successfully demonstrates the practical implementation of important Data Structures and Algorithms using C++ and SFML 3. It provides an interactive graphical environment that helps users understand complex algorithmic concepts through visual representation and animations.

The application combines two major components: Graph Visualization and Binary Search Tree Visualization. The graph module demonstrates Dijkstra's Shortest Path Algorithm, Prim's Minimum Spanning Tree Algorithm, and Kruskal's Minimum Spanning Tree Algorithm. The BST module supports insertion, searching, deletion, different traversal techniques, and calculation of tree height and diameter.

The project also demonstrates the practical use of important data structures such as vectors, queues, priority queues, and Disjoint Set Union (DSU). The graphical interface makes it easier to observe how algorithms process data and produce results.

Through this project, we gained practical experience in C++ programming, algorithm implementation, data structure management, graphical user interface development, and problem-solving.

Overall, the Graph Visualizer serves as an interactive learning tool that bridges the gap between theoretical knowledge and practical implementation. It also provides a foundation for future improvements by introducing more algorithms, advanced data structures, and enhanced visualization features.

17. REFERENCES

The following books, documentation, and online resources were referred to for understanding and implementing the concepts used in the Graph Visualizer project.

[1] Bjarne Stroustrup – *The C++ Programming Language*, Addison-Wesley.

[2] Thomas H. Cormen et al. – *Introduction to Algorithms*, MIT Press.

[3] Mark Allen Weiss – *Data Structures and Algorithm Analysis in C++*, Pearson.

[4] C++ Reference – https://en.cppreference.com/

[5] SFML Official Documentation – https://www.sfml-dev.org/documentation/

[6] CMake Official Documentation – https://cmake.org/documentation/

[7] GeeksforGeeks – https://www.geeksforgeeks.org/